

Title

1. Objective

Build a fully runnable local Proof of Concept demonstrating how business users can ask questions in plain English and receive SQL generated from a combination of structured metadata lookup and RAG retrieval. The POC should run locally with Python, SQLite and ChromaDB. Avoid cloud dependencies and external services.

2. End-to-End Architecture

User Question → NLP Analyzer → Entity/Metric Extraction → Metadata Repository Lookup → Relationship Resolution → RAG Retrieval → Context Builder → SQL Generator → Confidence Engine → Clarification Engine → User Approval.

3. Design Principles

Structured metadata must be stored in SQLite and retrieved deterministically. Business rules, glossary, governance, and semantic documentation must be stored in ChromaDB. Hybrid retrieval should combine keyword, BM25 and vector similarity.

4. Metadata Repository

Create semantic_entities, semantic_metrics, semantic_relationships, semantic_metric_mappings, semantic_source_mappings and workspace_config tables. Populate with Salesforce, Epicor, Priority ERP and SYSPRO examples.

5. ERP Sample Domains

Customer, Product, Order, Invoice, Supplier, Warehouse, Region, SalesPerson, Revenue, Margin, Profit, CustomerCount, OrderCount.

6. RAG Knowledge Base

Store business rules, glossary definitions, metric definitions, governance policies and calculation rules. Demonstrate chunking and embedding generation.

7. Chunking Strategies

Implement Fixed Chunking, Recursive Chunking and Metadata-Aware Chunking. Allow configuration to switch strategies. Display chunk counts and sizes.

8. Embeddings

Use sentence-transformers all-MiniLM-L6-v2. Generate embeddings for every chunk and store in ChromaDB. Show vector dimensions and collection statistics.

9. Retrieval

Implement similarity search, top-k retrieval, metadata filtering, threshold filtering and hybrid ranking. Display scores for each retrieved chunk.

10. NLP Analyzer

Extract entities, metrics, dimensions, filters and time periods from user questions. Examples: Revenue, Customer, Region, Month, Top 10.

11. Context Builder

Merge metadata lookup results with retrieved business rules and relationships. Build a single JSON context object passed to the SQL generator.

12. SQL Generation

Use deterministic templates. No external LLM required. Generate SQL for revenue analysis, profitability, trends, customer analysis and operational reporting.

13. Confidence Engine

Calculate confidence from metadata match, relationship coverage, similarity scores, metric resolution and question understanding.

14. Clarification Engine

When confidence is below threshold, generate clarification questions asking users to choose metrics, dimensions or filters.

15. Folder Structure

ai_dw_nlp_poc/data, metadata, db, vector_store, src/loaders, chunking, embeddings, retrieval, analyzer, metadata_lookup, context_builder, sql_generator, confidence, run_demo.py.

16. Local Execution

The entire project must run using: `pip install -r requirements.txt` and `python run_demo.py`. No OpenAI API keys required. No cloud databases required.

17. Demonstration Scenarios

Monthly Revenue by Customer, Top Customers by Profit, Revenue by Region, Product Margin Analysis, Orders by Supplier, Warehouse Performance, Customer Growth Trends.

18. Logging Requirements

Print every stage of execution: extracted entities, metadata matches, retrieved chunks, similarity scores, context object, SQL and confidence score.

19. Production Mapping

Explain how SQLite can later be replaced by PostgreSQL and ChromaDB can scale to enterprise semantic knowledge repositories while keeping the architecture unchanged.

20. Deliverables

Generate complete source code, sample metadata, sample ERP knowledge base, SQLite schema, ChromaDB setup, retrieval logic, context builder, SQL generator, confidence engine, README and runnable demo.